

Preguntes sobre Documentació Tècnica

David Juncosa, Javier Pedragosa i Guillem Riera

Heu de buscar com es fa una documentació tècnica en un ecosistema developer de backend. I fer aquest projecte segons la informació que heu buscat.

La documentació tècnica en backend és bàsicament explicar què fa el teu codi perquè els altres (o tu mateix d'aquí a 3 mesos) no s'hagin de trencar el cap llegint-ho.

Com es comenta el codi i bones praxis en Python. Com podem, si es pot fer, que aquesta documentació es generi de forma automàtica?

En Python el més habitual és comentar el codi utilitzant comentaris amb # i també docstrings, que són textos entre `""" """` que serveixen per explicar què fa una funció, classe o fitxer.

Algunes bones pràctiques són:

Utilitzar noms de variables clars, fer funcions petites i entenedores, no abusar dels comentaris inútils, explicar el perquè del codi, no només què fa; i seguir l'estil PEP8.

La documentació es pot generar automàticament amb eines com Sphinx o pydoc, que llegeixen els docstrings i creen documentació en HTML o PDF.

Identifica les diferències entre l'execució de codi al servidor i al client web.

El codi del client s'executa al navegador de l'usuari. Normalment és JavaScript, HTML i CSS. Serveix per mostrar la interfície i interactuar amb l'usuari.

El codi del servidor s'executa en una màquina remota. Aquest codi s'encarrega de processar dades, accedir a bases de dades, validar usuaris o gestionar la lògica de l'aplicació.

La principal diferència és que el client és la part visual i interactiva, mentre que el servidor gestiona la informació i la seguretat.

Reconeix els avantatges d'unir les dues tecnologies, frontend-backend, en el procés de desenvolupament de programes.

Unir frontend i backend permet crear aplicacions completes i funcionals.

Alguns avantatges són:

La separació de responsabilitats, un millor manteniment del codi, la possibilitat de treballar en equip, tenir aplicacions més escalables i la comunicació mitjançant APIs.

El frontend s'encarrega de la interfície i el backend de la lògica i les dades. Això fa que el desenvolupament sigui més ordenat.

Què és Flask? Com funciona i les seves principals característiques. Comparativa exhaustiva entre Flask i Django.

Flask és un framework de Python que serveix per crear aplicacions web i APIs de manera senzilla.

Funciona rebent peticions HTTP i retornant respostes. Les rutes es defineixen amb decoradors.

Característiques principals de Flask:

1. És lleuger i simple.
2. Té molta flexibilitat.
3. És fàcil d'aprendre.
4. Permet afegir extensions.

Django és un framework molt més complet. Ja inclou eines com autenticació, ORM, panell d'administració i sistema de plantilles.

Comparació:

1. Flask és més minimalista.
2. Django és més gran i estructurat.
3. Flask dona més llibertat.
4. Django és més ràpid per fer projectes grans.
5. Flask és ideal per APIs i projectes petits.
6. Django és millor per aplicacions complexes.

Anomeneu les principals llibreries que s'utilitzen en Flask. Per a què serveixen?

Algunes llibreries importants de Flask són:

1. Flask-SQLAlchemy: serveix per treballar amb bases de dades.
2. Flask-Login: gestiona l'autenticació d'usuaris.
3. Flask-WTF: facilita la creació de formularis.
4. Jinja2: motor de plantilles HTML.
5. Flask-RESTful: ajuda a crear APIs REST.
6. Flask-Migrate: gestiona migracions de bases de dades.

Aquestes extensions permeten ampliar les funcionalitats del framework.

Expliqueu, per exemple, una pàgina web molt senzilla cap a quin Framework us decantaríeu.

Per una pàgina web molt senzilla probablement utilitzaríem Flask.

Flask és més lleuger, fàcil de configurar i té menys complexitat. Per un projecte petit no cal tota l'estructura que ofereix Django.

En canvi, si el projecte hagués de créixer molt, tenir molts usuaris o funcionalitats complexes, seria millor Django.

En Python les proves unitàries com es realitzen? Busqueu informació i feu un exemple en codi.

Les proves unitàries serveixen per comprovar que petites parts del programa funcionen correctament.

En Python es poden fer amb el mòdul unittest:

A screenshot of a code editor with a dark background and light-colored text. The code is a Python script using the unittest module to test a simple addition function. It includes imports, a function definition, a test class, and a main execution block.

```
1 import unittest
2
3 def suma(a, b):
4     return a + b
5
6 class TestSuma(unittest.TestCase):
7
8     def test_suma(self):
9         self.assertEqual(suma(2, 3), 5)
10
11 if __name__ == "__main__":
12     unittest.main()
```

Què és una factoria? Feu un exemple en codi Python, dins d'un framework, i expliqueu com funcionaria.

Una factoria és un patró de disseny que serveix per crear objectes de forma organitzada.

En Flask s'utilitza molt el patró Application Factory.



```
1  from flask import Flask
2
3  def create_app():
4      app = Flask(__name__)
5
6      @app.route("/")
7      def inici():
8          return "Aplicació creada amb factoria"
9
10     return app
```

La funció `create_app()` crea i configura l'aplicació Flask.

Això permet separar configuracions, crear diferents versions de l'aplicació, facilitar les proves i organitzar millor el projecte, entre altres.